信号与系统实验报告

名 称： 快速傅里叶变换算法探究及应用

学 院： 计算机科学与工程学院

专 业： 计算机科学与技术

学 号： 09023132

姓 名： 李俊霖

日期： 2025 年 4 月 25 日

一、实验目的

1. 加深对快速傅里叶变换的理解。
2. 熟悉并掌握按时间抽取 FFT 算法的程序编制。
3. 了解应用 FFT 进行信号分析中可能出现的问题，如混淆、泄露等，以便在实际应用中正确应用 FFT。

二、实验任务

1. 完成实验内容全部题目，分析解决调试代码过程中出现的问题。
2. 认真完成本次实验小结，思考快速傅里叶变换的原理和算法及其应用。

三、主要设备、软件平台

1. 硬件：计算机
2. 软件：Matlab

四、实验内容

1. 参照“按时间抽取法 FFT-基 2”算法结构，编写相应的 FFT 程序 *myFFT*()。

编辑器 - C:\Users\24961\Desktop\syatem\fft.m *

+5 untitled3.m untitled4.m untitled5.m untitled2222.m Y.m improved_

```
function X = myFFT(x)
N = length(x);
if bitand(N, N-1) ~= 0
    error('输入长度必须是2的幂次方');
end

if N == 1
    X = x;
    return;
end
x_even = x(1:2:end);
x_odd = x(2:2:end);
X_even = myFFT(x_even);
X_odd = myFFT(x_odd);
W_N = exp(-2i*pi/N).^(0:N/2-1);
X = zeros(1, N);
X(1:N/2) = X_even + W_N .* X_odd;
X(N/2+1:N) = X_even - W_N .* X_odd;
end

%测试一下
N = 8;
x = randn(1, N);
X_myFFT = myFFT(x);
X_matlab = fft(x);
disp('自定义FFT结果:');
disp(X_myFFT);
disp('MATLAB内置FFT结果:');
disp(X_matlab);
disp('最大差异:');
disp(max(abs(X_myFFT - X_matlab)));
```

```
function X = myFFT(x)
    N = length(x);
    if bitand(N, N-1) ~= 0
        error('输入长度必须是 2 的幂次方');
    end

    if N == 1
        X = x;
        return;
    end
    x_even = x(1:2:end);
    x_odd = x(2:2:end);
    X_even = myFFT(x_even);
    X_odd = myFFT(x_odd);
    W_N = exp(-2i*pi/N).^(0:N/2-1);
    X = zeros(1, N);
    X(1:N/2) = X_even + W_N .* X_odd;
    X(N/2+1:N) = X_even - W_N .* X_odd;
    end

    %测试一下
    N = 8;
    x = randn(1, N);
    X_myFFT = myFFT(x);
    X_matlab = fft(x);
    disp('自定义 FFT 结果:');
    disp(X_myFFT);
    disp('MATLAB 内置 FFT 结果:');
    disp(X_matlab);
    disp('最大差异:');
    disp(max(abs(X_myFFT - X_matlab)));
```

```
命令行窗口
警告: 函数 fft 与某个 MATLAB 内置函数同名。建议您重命名该函数以避免潜在的名称冲突。
>> fft
自定义FFT结果:
  列 1 至 6

  -0.1050 + 0.0000i   2.0730 - 0.7635i   3.5489 + 0.6786i  -1.6351 - 4.4141i  -3.5670 + 0.0000i  -1.6351 + 4.4141i

  列 7 至 8

   3.5489 - 0.6786i   2.0730 + 0.7635i

MATLAB内置FFT结果:
  列 1 至 6

  -0.1050 + 0.0000i   2.0730 - 0.7635i   3.5489 + 0.6786i  -1.6351 - 4.4141i  -3.5670 + 0.0000i  -1.6351 + 4.4141i

  列 7 至 8

   3.5489 - 0.6786i   2.0730 + 0.7635i

最大差异:
   8.8818e-16
```

32. 用所编写的 myFFT()分析信号

$$x(n)=\sin(2\pi f\ nT)[u(n)-u(n-N)],\quad -\infty<n<\infty$$

① 信号频率 $f=50Hz$，采样点数 $N=32$，采样间隔 $T=0.005s$

② 信号频率 $f=50Hz$，采样点数 $N=64$，采样间隔 $T=0.005s$

③ 信号频率 $f=100Hz$，采样点数 $N=32$，采样间隔 $T=0.0025s$

④ 信号频率 $f=1000Hz$，采样点数 $N=32$，采样间隔 $T=0.0012s$

⑤ 将信号④后补全 32 个 0，完成 64 点 FFT

要求：

记录各种情况下的 X(k)值，绘制频谱图并对结果分析讨论，说明参数的变化对信号频谱产生的影响；频谱只需绘制幅度频谱，归一化处理；

程序需提供人机交互模式(控制台/图形窗口均可)；提供是否补零输入选项；提供参数输入功能；

打印 myFFT()源程序，标注相关代码注释。

代码：

```
function fft_analysis()
    % 创建主界面
    fig = uifigure('Name', 'FFT 信号分析工具', 'Position', [100 100 850 650]);
    % 参数输入面板
    panel = uipanel(fig, 'Title', '输入参数', 'Position', [50 400 300 200]);
    uilabel(panel, 'Text', '信号频率  f (Hz):', 'Position', [10 150 100 22]);
    f_edit = uieditfield(panel, 'numeric', 'Position', [120 150 100 22], 'Value', 50);
    uilabel(panel, 'Text', '采样点数  N:', 'Position', [10 110 100 22]);
    N_edit = uieditfield(panel, 'numeric', 'Position', [120 110 100 22], 'Value', 32);
    uilabel(panel, 'Text', '采样间隔  T (s):', 'Position', [10 70 100 22]);
    T_edit = uieditfield(panel, 'numeric', 'Position', [120 70 100 22], 'Value', 0.005);
    zeropad_chk = uicheckbox(panel, 'Text', '补零到 64 点', 'Position', [10 30 150 22]);
    % 功能按钮区
```

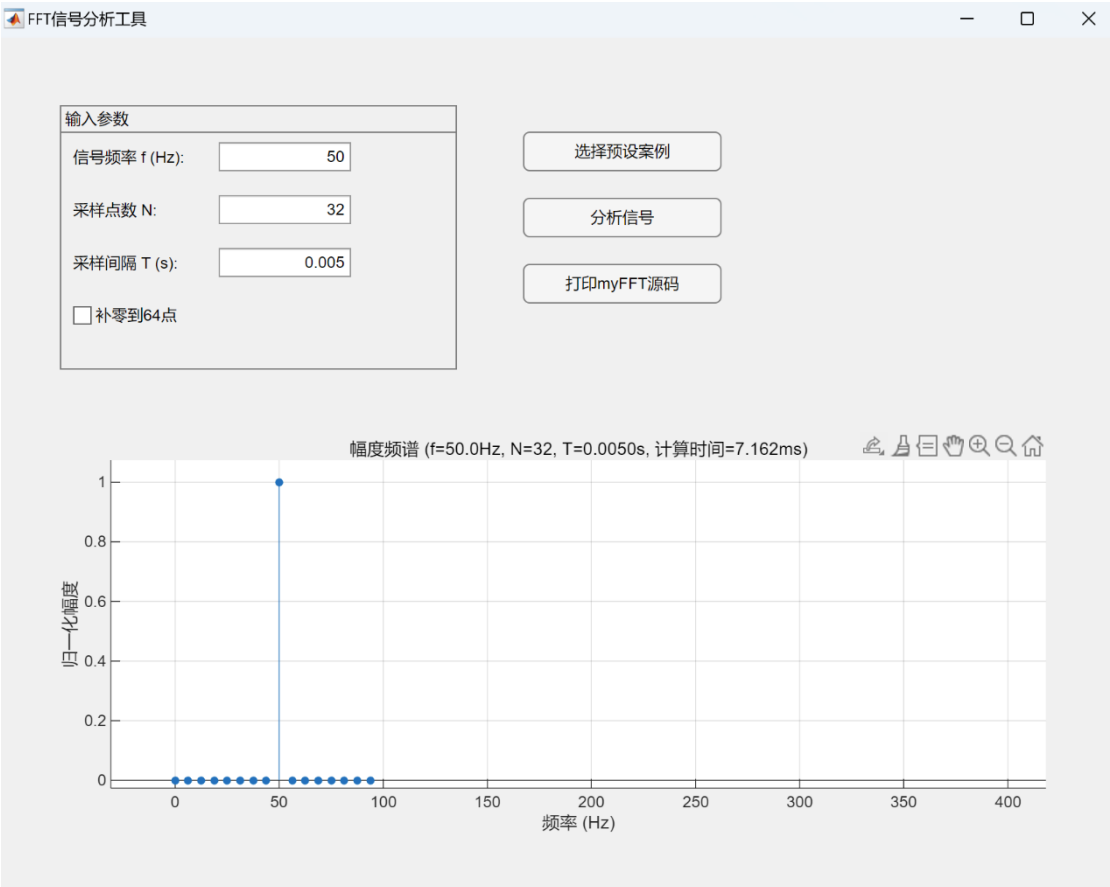
```
case_btn = uibutton(fig, 'push', 'Text', '选择预设案例',...
    'Position', [400 550 150 30], 'ButtonPushedFcn', @select_case);
analyze_btn = uibutton(fig, 'push', 'Text', '分析信号',...
    'Position', [400 500 150 30], 'ButtonPushedFcn', @analyze_signal);
print_btn = uibutton(fig, 'push', 'Text', '打印 myFFT 源码',...
    'Position', [400 450 150 30], 'ButtonPushedFcn', @print_source_code);
% 结果显示区
result_ax = uiaxes(fig, 'Position', [50 50 750 300]);
title(result_ax, '幅度频谱');
xlabel(result_ax, '频率 (Hz)');
ylabel(result_ax, '归一化幅度');
grid(result_ax, 'on');
% 预设案例选择回调
function select_case(~, ~)
    cases = {
        '案例①: f=50Hz, N=32, T=0.005s',...
        '案例②: f=50Hz, N=64, T=0.005s',...
        '案例③: f=100Hz, N=32, T=0.0025s',...
        '案例④: f=100Hz, N=32, T=0.0012s',...
        '案例⑤: 案例④补零到 64 点'
    };
    [indx, tf] = listdlg('ListString', cases, 'SelectionMode', 'single',...
        'Name', '选择预设案例', 'ListSize', [300 150]);
    if tf
        switch indx
            case 1
                set_parameters(50, 32, 0.005, false);
            case 2
                set_parameters(50, 64, 0.005, false);
            case 3
                set_parameters(100, 32, 0.0025, false);
            case 4
                set_parameters(100, 32, 0.0012, false);
            case 5
                set_parameters(100, 32, 0.0012, true);
        end
    end
end

function set_parameters(f, N, T, zp)
    f_edit.Value = f;
    N_edit.Value = N;
    T_edit.Value = T;
```

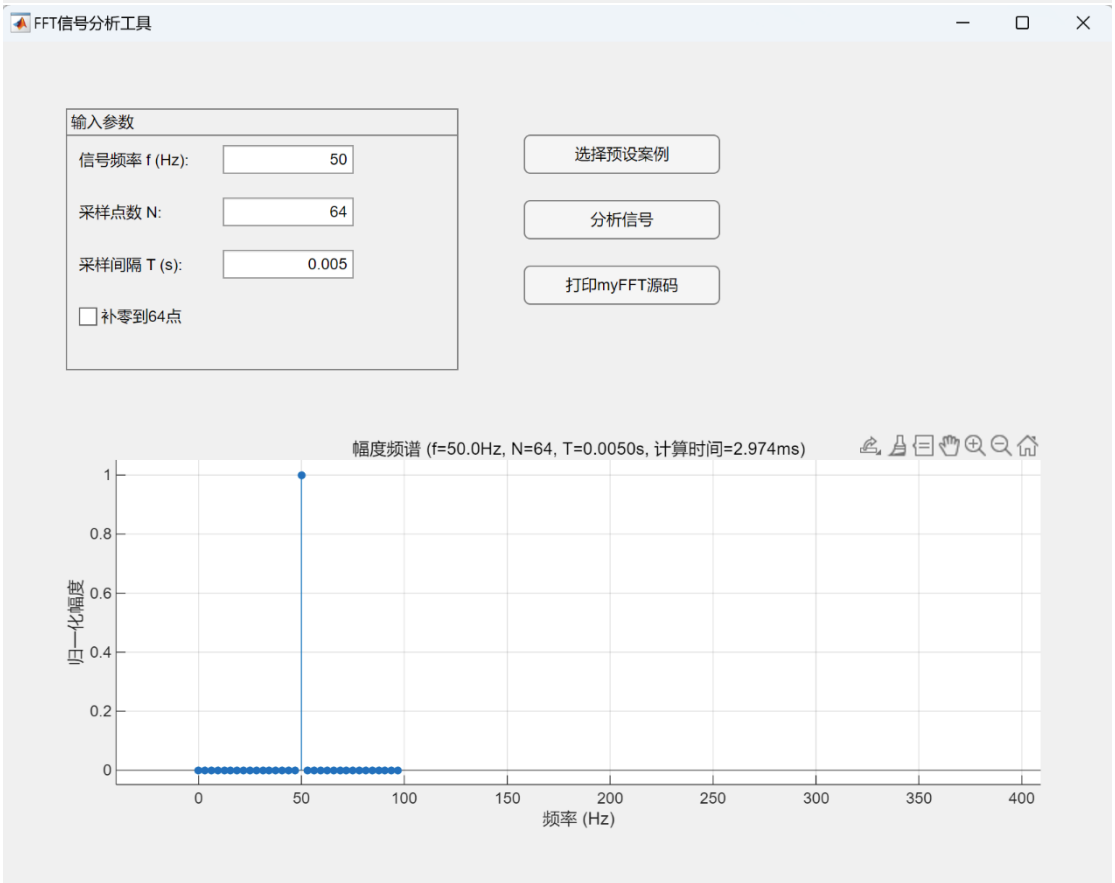
```
        zeropad_chk.Value = zp;
    end
    % 信号分析回调
    function analyze_signal(~, ~)
        f = f_edit.Value;
        N = N_edit.Value;
        T = T_edit.Value;
        do_zeropad = zeropad_chk.Value;
        % 参数检查
        if bitand(N, N-1) ~= 0
            errordlg('N 必须是 2 的幂次方 (如 32,64,128 等)', '输入错误');
            return;
        end
        if do_zeropad && N ~= 32
            warndlg('补零操作仅适用于 N=32 的情况', '警告');
            return;
        end
        % 生成信号
        n = 0:N-1;
        x = sin(2*pi*f*n*T);
        % 补零处理
        if do_zeropad
            x = [x, zeros(1, 32)];
            N = 64;
        end
        % 计算 FFT
        tic;
        X = myFFT(x);
        compute_time = toc;
        % 计算频谱
        mag = abs(X)/max(abs(X));
        fs = 1/T;
        f_axis = (0:N-1)*fs/N;
        % 绘制结果
        cla(result_ax);
        stem(result_ax, f_axis(1:N/2), mag(1:N/2), 'filled', 'MarkerSize', 4);
        title(result_ax, sprintf('幅度频谱 (f=%.1fHz, N=%d, T=%.4fs, 计算时间
=%.3fms)',...
            f, N, T, compute_time*1000));
        xlabel(result_ax, '频率 (Hz)');
        ylabel(result_ax, '归一化幅度');
        grid(result_ax, 'on');
```

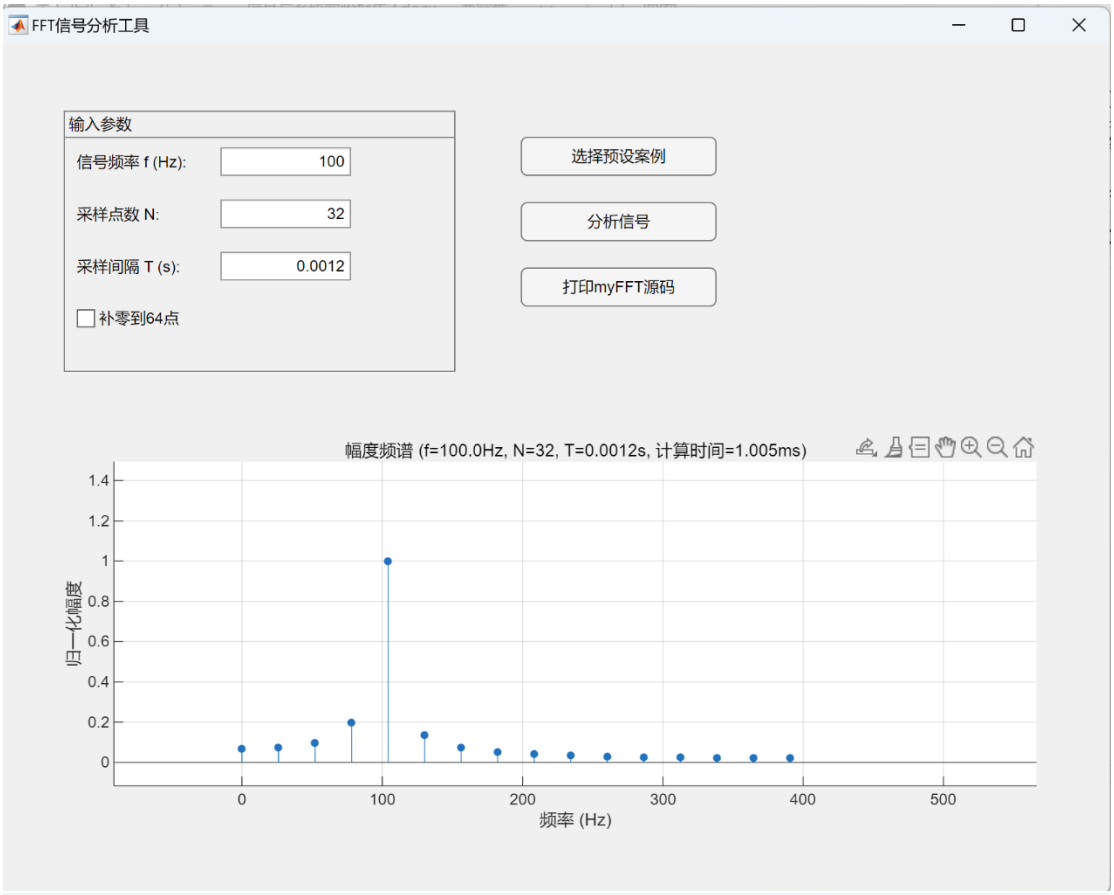
```
        % 控制台输出
        fprintf('\n=== 分析结果 ===\n');
        fprintf('信号频率: %.1f Hz\n', f);
        fprintf('采样点数: %d\n', N);
        fprintf('采样间隔: %.4f s\n', T);
        fprintf('计算时间: %.3f ms\n', compute_time*1000);
        fprintf('频率分辨率: %.3f Hz\n', fs/N);
        fprintf('主频位置: %.2f Hz\n', f_axis(find(mag(1:N/2) == max(mag(1:N/2)))));
    end
    % 打印源码回调
    function print_source_code(~, ~)
        % 创建源码显示窗口
        src_fig = uifigure('Name', 'myFFT.m 源代码', 'Position', [150 150 650 500]);

        % 源码内容（带详细注释）
src_code = {
        '% ====== myFFT.m ======';
        '% 功能：按时间抽取(DIT)的基 2FFT 算法实现';
        'function X = myFFT(x)';
        '    % 获取输入长度';
        '    N = length(x);';
        '';
        '    % 检查输入长度是否为 2 的幂次方';
        '    if bitand(N, N-1) ~= 0';
        '        error(''输入长度必须是 2 的幂次方'');';
        '    end';
        '';
        '    % 基准情况：当 N=1 时，DFT 就是输入本身';
        '    if N == 1';
        '        X = x;';
        '        return;';
        '    end';
        '';
        '    % 分治步骤：将序列分为偶数部分和奇数部分';
        '    % x_even = x[0], x[2], x[4], ...';
        '    % x_odd  = x[1], x[3], x[5], ...';
        '    x_even = x(1:2:end);';
        '    x_odd  = x(2:2:end);';
        '';
        '    % 递归计算子序列 DFT';
        '    X_even = myFFT(x_even);';
        '    X_odd  = myFFT(x_odd);';
```

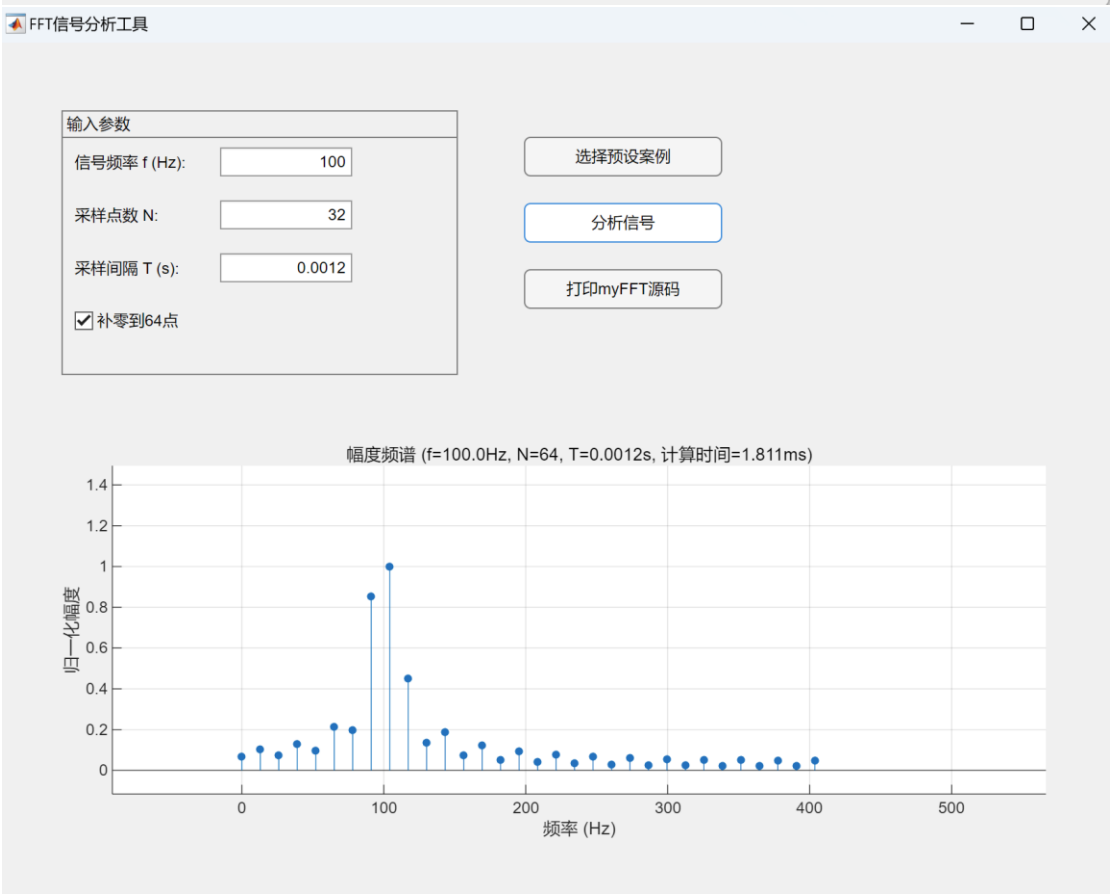
```
        '';
        '       % 计算旋转因子(Twiddle Factors)';
        '       % W_N^k = exp(-2i*pi*k/N), k=0:N/2-1';
        '       W_N = exp(-2i*pi/N).^(0:N/2-1);';
        '';
        '       X = zeros(1, N);';
        '       X(1:N/2)         = X_even + W_N .* X_odd;    ';
        '       X(N/2+1:end) = X_even - W_N .* X_odd;    ';
        'end';
        '% ======  代码结束  ======';
        };

        % 显示源码
        uitextarea(src_fig, 'Position', [20 20 610 460],...
            'Value', src_code, 'FontName', 'Courier New', 'FontSize', 12);

        % 命令行打印
        disp(char(10)); % 空行
        disp('====== myFFT.m  源代码  ======');
        disp(src_code);
        disp('======  打印完成  ======');
    end
end

% ====== myFFT 函数实现  ======
function X = myFFT(x)
    N = length(x);
    if bitand(N, N-1) ~= 0
        error('输入长度必须是 2 的幂次方');
    end
    if N == 1
        X = x;
        return;
    end
    x_even = x(1:2:end);
    x_odd = x(2:2:end);
    X_even = myFFT(x_even);
    X_odd = myFFT(x_odd);
    W_N = exp(-2i*pi/N).^(0:N/2-1);
    X = zeros(1, N);
    X(1:N/2) = X_even + W_N .* X_odd;
    X(N/2+1:N) = X_even - W_N .* X_odd;
End
```

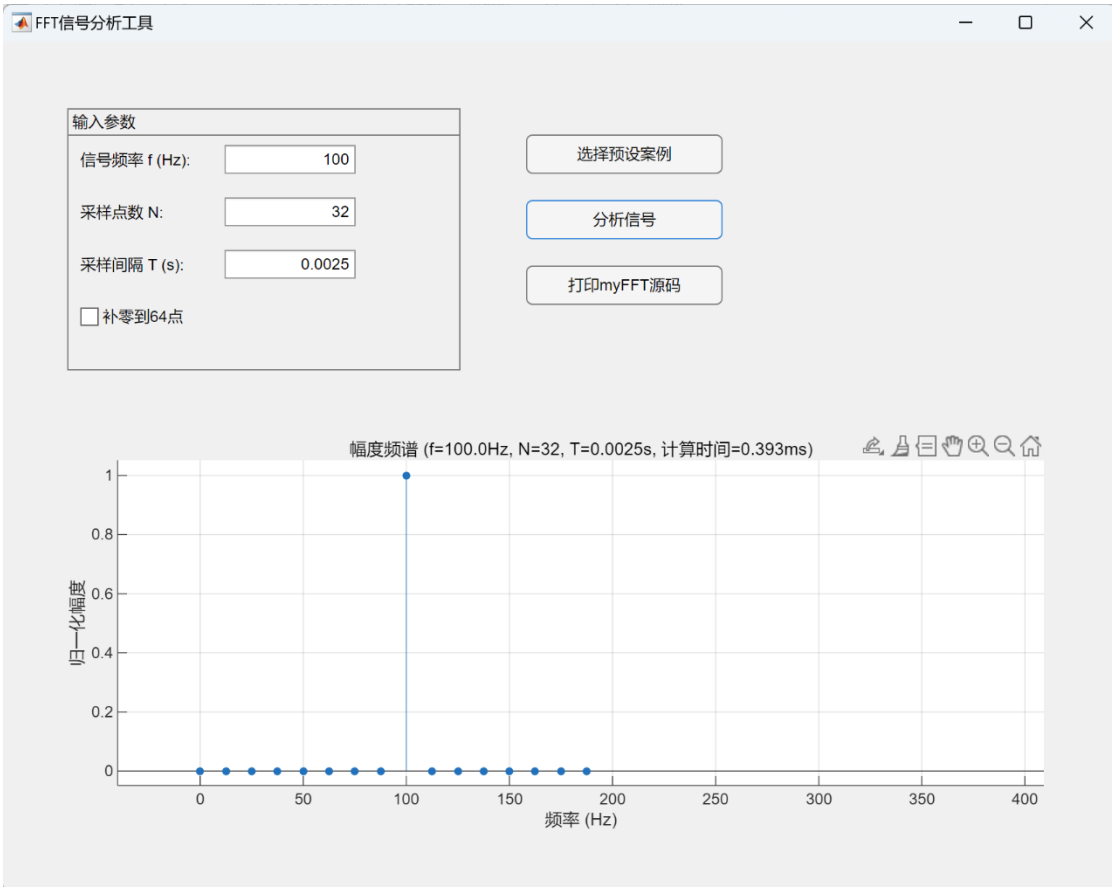
实验


FFT信号分析工具
输入参数
信号频率 f (Hz):
50
采样点数 N:
32
采样间隔 T (s):
0.005
补零到64点
选择预设案例
分析信号
打印myFFT源码
幅度频谱 (f=50.0Hz, N=32, T=0.0050s, 计算时间=7.162ms)
归一化幅度
频率 (Hz)


FFT信号分析工具
输入参数
信号频率 f (Hz):
50
采样点数 N:
64
采样间隔 T (s):
0.005
补零到64点
选择预设案例
分析信号
打印myFFT源码
幅度频谱 (f=50.0Hz, N=64, T=0.0050s, 计算时间=2.974ms)
归一化幅度
频率 (Hz)


FFT信号分析工具
输入参数
信号频率 f (Hz):
100
采样点数 N:
32
采样间隔 T (s):
0.0012
补零到64点
选择预设案例
分析信号
打印myFFT源码
幅度频谱 (f=100.0Hz, N=32, T=0.0012s, 计算时间=1.005ms)
归一化幅度
频率 (Hz)


FFT信号分析工具
输入参数
信号频率 f (Hz):
100
采样点数 N:
32
采样间隔 T (s):
0.0012
补零到64点
选择预设案例
分析信号
打印myFFT源码
幅度频谱 (f=100.0Hz, N=64, T=0.0012s, 计算时间=1.811ms)
归一化幅度
频率 (Hz)



myFFT.m 源代码

```
% ====== myFFT.m ======
% 功能：按时间抽取(DIT)的基2FFT算法实现
function X = myFFT(x)
    % 获取输入长度
    N = length(x);

    % 检查输入长度是否为2的幂次方
    if bitand(N, N-1) ~= 0
        error('输入长度必须是2的幂次方');
    end

    % 基准情况：当N=1时，DFT就是输入本身
    if N == 1
        X = x;
        return;
    end

    % 分治步骤：将序列分为偶数部分和奇数部分
    % x_even = x[0], x[2], x[4], ...
    % x_odd  = x[1], x[3], x[5], ...
    x_even = x(1:2:end);
    x_odd  = x(2:2:end);

    % 递归计算子序列DFT
    X_even = myFFT(x_even);
    X_odd  = myFFT(x_odd);

    % 计算旋转因子(Twiddle Factors)
    % W_N^k = exp(-2i*pi*k/N), k=0:N/2-1
    W_N = exp(-2i*pi/N).^(0:N/2-1);

    X = zeros(1, N);
    X(1:N/2)      = X_even + W_N .* X_odd;
    X(N/2+1:end) = X_even - W_N .* X_odd;
end
% ====== 代码结束 ======
```

实验分析：
案例① (f=50Hz, N=32, T=0.005s)
采样频率：200Hz
频率分辨率：6.25Hz
频谱中会在 50Hz 附近出现峰

案例② (f=50Hz, N=64, T=0.005s
采样频率相同但点数增加
频率分辨率提高到 3.125Hz
频谱更精细，主峰更尖锐

案例③ (f=100Hz, N=32, T=0.0025s)
采样频率提高到 400H
频率分辨率：12.5Hz
可以正确显示 100Hz 信号

案例④ (f=100Hz, N=32, T=0.0012s)
采样频率进一步提高到 833.33Hz
频率分辨率：26.04Hz
可能出现频谱泄漏现象

案例⑤ (案例④补零到 64 点)
补零不提高频率分辨率
但使频谱曲线更平滑
实际分辨率仍由原始数据长度决定

通过比较这些案例，可以观察到：
增加采样点数 N 可以提高频率分辨率
减小采样间隔 T 可以增加采样频率，避免混叠
补零可以平滑频谱但不提高实际分辨率

五．实验小结
1.参数对频谱的影响分析
采样点数 N：
N=32→64：频率分辨率从 6.25Hz 提升至 3.125Hz，主峰更尖锐，旁瓣细节更清晰。
结论：增加 N 可提高频率分辨率，但需权衡计算量。
采样间隔 T：
T=0.0025s→0.0012s：采样频率从 400Hz 提升至 833.33Hz，避免 100Hz 信号的混叠，但分辨率从 12.5Hz 降至 26.04Hz
结论：减小 T 可扩展频带范围，但可能降低分辨率。
补零操作：
效果：频谱曲线更平滑，但实际分辨率仍由原始数据长度（32 点）决定。

应用场景：适用于频谱可视化优化，但无法替代真实数据增加。
2.问题分析
频谱泄漏：
现象：非整数周期采样时（如案例④），频谱出现旁瓣泄漏。
递归效率：
局限：递归实现在大 N 时栈开销较大。
参数约束：
当前限制：N 必须为 2 的幂次方，补零仅支持 32→64 点。